- *Volume of data*—The source code you write as a developer and all the libraries, packages, and modules you build your service on top of make up a large part of the data. Then, there are contextual datasets, including issue trackers (GitHub, GitLab, etc.) as well as road maps and user input and feedback, such as interviews or recordings. Then, there is documentation, both related to the source code and standalone project or product docs, and APIs, with their actions and data types based on a schema. All of that data serves as input, and as developers, we tend to try to keep all of these connections between the various artifacts in our heads. This is increasingly challenging, the more layers of abstractions we use.
- *Testing*—Many things changed with the introduction of cloud offerings, and I don't necessarily mean cloud-based IDEs; instead, I mean where and how software is tested. Now, we will keep doing unit testing (depending on the programming language, that's a more or less important aspect of reliable and maintainable code), but where should we do integration and end-to-end tests? One hypothesis is to test in production and mitigate impact via canaries, A/B testing, and feature flags (as offered by LaunchDarkly [https://launchdarkly.com/]) or Amazon CloudWatch Evidently (https://aws.amazon.com/blogs/aws/cloudwatch-evidently/).
- *Distributed systems*—Microservices are distributed systems and, as such, rely on network communication. No matter whether you're using HTTP, gRPC, or WebSockets, you need to understand the communication behavior, the payload (encoding, format, etc.), and how to explore what a remote API does when it experiences load. This means being forced to, on the one hand, deal with leaky abstractions (implementation details shining through an abstraction layer) and, on the other hand, detect and handle timeouts, retries, and throttling. Remember that your network(s) are likely not reliable or secure (https://architecturenotes.co/fallacies-of-distributed-systems/).

9.2.2 Tooling

The developer productivity tools we will have a closer look at in the following subsections are generally based on or use OpenTelemetry and provide deep IDE integrations, typically into VS Code. Increasingly, companies have begun to open source their (usually OpenTelemetry-based) developer productivity–related tooling, including eBay, which made an interesting framework called *flow telemetry* (https://github.com/eBay/flow-telemetry) available. Note that the tools discussed in this section are rapidly evolving, and you can expect changes in the functionality or consolidations with other tools or platforms.

Digma

Digma (https://digma.ai/) is positioned as a *continuous feedback pipeline* (which seems to be a marketing term, as I haven't found a formal definition), with an analysis backend and IDE plug-in. Its goal is to continually analyze observability sources and